Collibra Usage Analytics API: A Hands-On Guide

Oct 19, 2026 · @Sajji

Introduction

By the end of this guide you will be able to pull your own Collibra Usage Analytics data with plain REST calls: what content gets visited, which assets matter, who is active, and how adoption is trending. Every example runs in Postman (or curl, Insomnia, Bruno, or a script) with no extra software.

Collibra's **Usage Analytics** app (under *Insights*) collects every visit to an asset, domain, community, dashboard and diagram, plus every user sign-in. The screens in the app are drawn from two REST services you can call yourself:

- `/rest/usageAnalyticsUsage/v1` answers *what was used*: visits by content type, top items, trends, and per-asset detail.
- `/rest/usageAnalyticsUsers/v1` answers *who used it*: active, inactive and new users, top users, usage rate, retention and license types.

The guide is built as a short course. You set up Postman once, learn the shape the API shares across all its endpoints, then work through four lessons and a set of recipes. Each lesson has a goal, the exact request, an abridged real response, and a *Try it* exercise.

You want to know…	Go to
How many visits each content type got this month	Lesson 1
Whether a specific asset is being used, and by whom	Lesson 2
How many people are active, and whether they come back	Lesson 3
The same numbers for one team, community or asset type	Lesson 4
A monthly report, a period-over-period comparison, or full row-level data	Putting it together

Before you begin

You need four things before your first call.

- [ ] **Usage Analytics enabled** on your Collibra environment. If *Insights > Usage Analytics* is missing from the main menu, ask your Collibra administrator.
- [ ] **A Collibra user with the Usage Analytics (Insights) permission.** In Collibra this is granted through a global role, such as the out-of-the-box *Insights* role. Without it every call below returns `403 Forbidden`.
- [ ] **Your environment URL**, for example `https://your-company.collibra.com`. This guide writes it as `{{baseUrl}}`.
- [ ] **An API client.** This guide uses Postman; the desktop app and the web app both work.

Important: these are internal APIs. The Usage Analytics services power Collibra's own Usage Analytics screens. They are not listed in Collibra's public REST API documentation, and Collibra can change or remove them in any release without notice. Treat anything you build on them as best-effort, pin your assumptions in one place (a Postman environment, a config file), and re-test after each Collibra upgrade.

Good practice for usage data. Responses can contain people's names and activity. Use a dedicated service account where your policy allows, store credentials only in Postman environment variables marked *secret*, and share exports in line with your organization's privacy rules. Every endpoint in this guide is read-only: it never creates, changes or deletes anything in Collibra.

Step 1: Set up Postman

This step takes about ten minutes and you do it once. At the end you will have a working collection and a green first response.

1.1 Create an environment

In Postman, open **Environments > +** and name it after your Collibra instance (for example *Collibra Production*). Add these variables:

Variable	Example value	Type	Used for
`baseUrl`	`https://your-company.collibra.com`	default	Every request URL. No trailing slash.

Variable	Example value	Type	Used for
`username`	`svc-usage-reader`	default	Basic authentication
`password`	(your password)	secret	Basic authentication
`usage`	`{{baseUrl}}/rest/usageAnalyticsUsage/v1`	default	Shortcut for the content service
`users`	`{{baseUrl}}/rest/usageAnalyticsUsers/v1`	default	Shortcut for the users service
`startDate`	`2026-09-01`	default	Start of the reporting window
`endDate`	`2026-09-30`	default	End of the reporting window (inclusive)

Save, then pick the environment from the drop-down at the top right of Postman.

1.2 Create a collection with shared authentication

1. Click **Collections** > **+** and name it *Collibra Usage Analytics*.
2. Open the collection's **Authorization** tab and choose **Basic Auth**.
3. Set *Username* to `{{username}}` and *Password* to `{{password}}`.
4. Save. Every request you add to this collection inherits the setting (*Auth Type: Inherit auth from parent*).

Basic authentication sends your credentials on each call, so there is no session or CSRF token to manage. If your organization signs in through SSO only, ask your administrator for a local service account for API use.

1.3 Make your first call

The simplest endpoint tells you when Collibra last refreshed the Usage Analytics data. Add a request to the collection:

```
GET {{usage}}/lastRefreshed
Accept: application/json
```

Click **Send**. You should see `200 OK` and a body like this:

```
{
  "lastRefreshDateTime": "2026-10-19T16:00:01.721588Z"
}
```

That timestamp matters for every report you run. Usage Analytics is aggregated periodically, not in real time, so activity after this time is not in the numbers yet. A safe habit is to end your date ranges on *yesterday*.

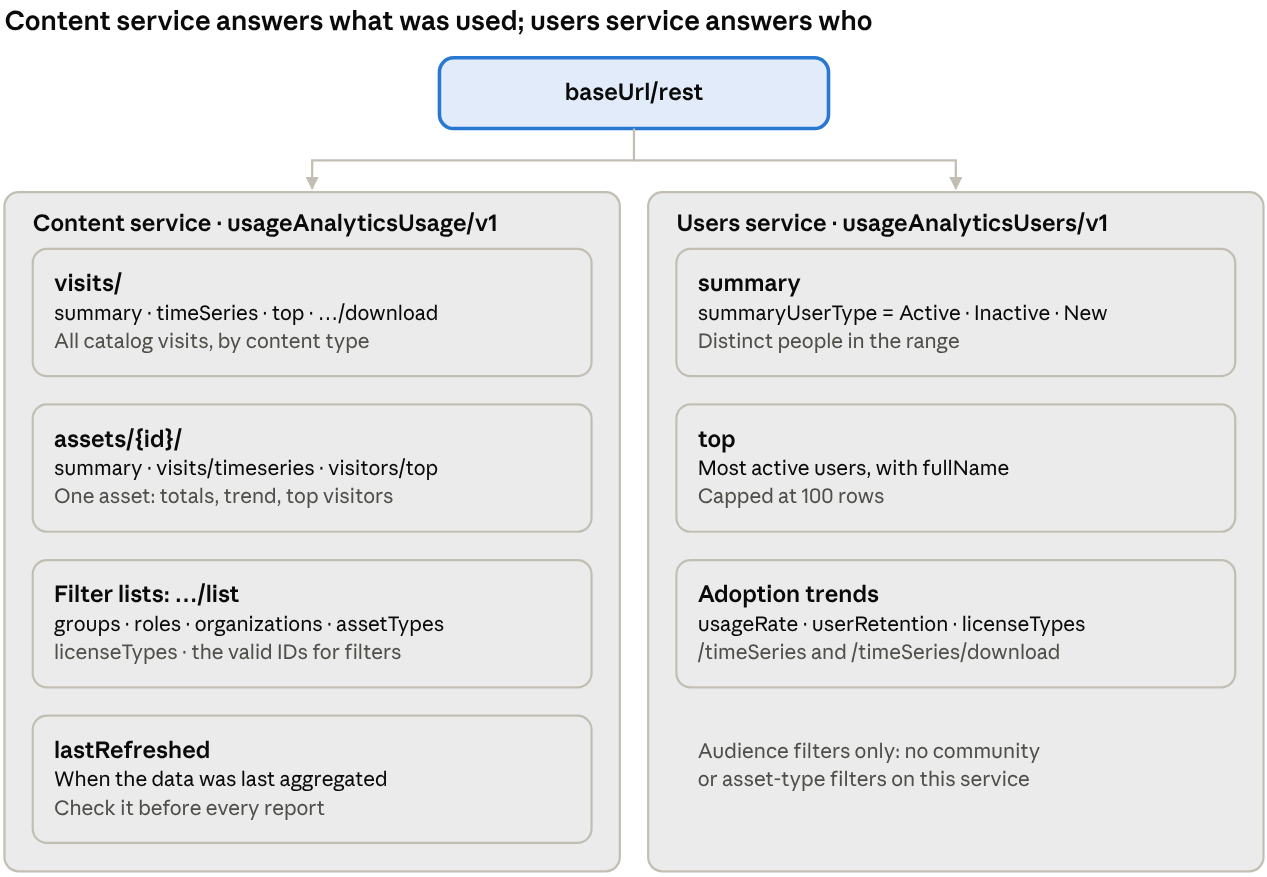
If you see…	It means…
`200 OK` with a timestamp	You are set up. Continue to the next section.
`401 Unauthorized`	The username or password is wrong, or the environment is not selected.
`403 Forbidden`	You are signed in, but the user lacks the Usage Analytics (Insights) permission.
`404 Not Found`	Check `baseUrl` for typos or a trailing slash. If it is right, Usage Analytics may not be enabled.

The same call in curl, for scripting:

```
curl -s -u "$COLLIBRA_USER:$COLLIBRA_PASS" \
  -H "Accept: application/json" \
  "https://your-
company.collibra.com/rest/usageAnalyticsUsage/v1/lastRefreshed"
```

How the API is shaped

Once you know three patterns, every endpoint in this guide reads the same way: all calls are `GET`, all options are query parameters, and most responses are a `results` array.



Usage Analytics endpoint map · 2 services, 7 endpoint families

Pick the service by the question (what versus who), then the endpoint family by the kind of answer you need.

Pattern 1: the four kinds of endpoint

Path ends in…	Returns	Example
`/summary`	One total per category for the whole date range	Visits per content type
`/timeSeries`	One row per time bucket (and category)	Visits per week
`/top`	A ranked list, capped at 100 rows	Most-visited assets
`/timeSeries/download`	A CSV file with one row per item (or user) per bucket, no 100-row cap	Every asset visited this quarter

There are also small `/list` endpoints that return the valid values for filters (Lesson 4).

Pattern 2: the shared query parameters

Almost every endpoint accepts the same set of parameters. Dates are `YYYY-MM-DD` and both ends are inclusive.

Parameter	Values	Required	Notes
`startDate`	`2026-09-01`	Yes	First day counted
`endDate`	`2026-09-30`	Yes	Last day counted. Use yesterday or earlier.
`granularity`	`Day`, `Week`, `Month`, `Quarter`	On `/timeSeries` and `/top`	Bucket size. `usageRate` does not accept `Day`.
`excludeAdmin`	`true`, `false`	No	Drops visits by administrators
`excludeDisabledUser`	`true`, `false`	No	Drops visits by users who are now disabled
`limit`	`1` to `100`	On `/top` and `/list`	Rows returned
`userGroupIds`	Group UUID	No	Repeatable. Filters to people in these groups.
`userRoleIds`	Role UUID	No	Repeatable. Filters to people holding these roles.
`userLicenseTypes`	`Creator`, `Consumer`, …	No	Repeatable. Filters by license type.
`organizationIds`	Community or domain UUID	Content service only	Repeatable. Narrows visits to this part of the catalog.
`assetTypeIds`	Asset type UUID	Content service only	Repeatable. Narrows visits to these asset types.

Repeating a parameter means OR. To filter on two groups, send the parameter twice: `?userGroupIds=<id-1>&userGroupIds=<id-2>`. In Postman, add two rows with the same key on the **Params** tab.

Pattern 3: the response envelope

JSON responses wrap their rows in `results`. Time-series rows always carry the bucket they belong to and the part of the bucket that falls inside your range:

```
{
  "results": [
    {
      "bucketStartDate": "2026-09-28",
      "bucketEndDate": "2026-10-04",
      "startDate": "2026-09-28",
      "endDate": "2026-10-04",
      "visitType": "Asset",
      "count": 1
    }
  ]
}
```

Two things trip people up here:

- **Buckets are calendar buckets.** A `Week` bucket runs Monday to Sunday, so the first and last buckets of your range can be partial. `startDate` and `endDate` on the row tell you the part that was counted.
- **Empty buckets are omitted.** A week with no visits has no row. Fill the gaps with zero yourself before charting.

All response examples in this guide are abridged to the fields you need; real responses may include more.

Lesson 1: What content is being used?

Goal: answer "how much was the catalog visited, and what did people look at?" in three calls: a summary, a trend and a top list.

1a. Visit totals by content type

```
GET {{usage}}/visits/summary?startDate={{startDate}}&endDate={{endDate}}
```

```
{
  "results": [
    { "visitType": "Asset",     "count": 46 },
    { "visitType": "Domain",    "count": 17 },
    { "visitType": "Community", "count": 11 },
    { "visitType": "Dashboard", "count": 72 },
    { "visitType": "Diagram",   "count": 5 }
  ]
}
```

Read it like this: each row is one content type; `count` is the number of page visits in the range. Add the five counts for total visits (151 here). A visit is a page view, not a person: one user opening the same asset five times counts five.

1b. Visits over time

Add `granularity` and switch to the `timeSeries` path:

```
GET {{usage}}/visits/timeSeries?startDate={{startDate}}&endDate=
{{endDate}}&granularity=Week
```

You get one row per week per content type, in the envelope shown in Pattern 3. To chart it, group rows by `bucketStartDate` and put each `visitType` in its own column.

1c. Most-visited items

```
GET {{usage}}/visits/top?startDate={{startDate}}&endDate=
{{endDate}}&visitType=Asset&limit=10&granularity=Day
```

```
{
  "results": [
    { "id": "72355264-0610-453a-a0c3-47fb927f3139", "label": "Customer
Analytics", "count": 16 },
    { "id": "01a0ed69-9bfe-702a-861b-239f56faa995", "label": "Commit",
"count": 3 },
    { "id": "01992f7f-d668-77bb-8b29-ff2be663a949", "label": "Healthcare -
Clinical Decision Support", "count": 3 }
  ]
}
```

`visitType` picks what is ranked: `Asset`, `Domain`, `Community`, `Dashboard` or `Diagram`. `id` is the item's Collibra UUID, so you can open it at `{{baseUrl}}/asset/<id>` or look up more detail with `GET {{baseUrl}}/rest/2.0/assets/<id>`.

Try it

1. Change `visitType` to `Dashboard`. Which dashboard is used most?
2. Run 1a again with `excludeAdmin=true`. How much of your traffic came from administrators?
3. Set `limit` to `100`, the maximum. If your catalog has more visited items than that, use the download endpoint in *Putting it together* for the complete list.

Lesson 2: Is this asset being used, and by whom?

Goal: for one asset, get its all-time totals, its visit trend and its top visitors. This is the call set behind questions like "can we retire this report?" or "who should we tell before we change this term?"

You need the asset's UUID. Take it from Lesson 1c, from the asset page URL in Collibra (`/asset/<id>`), or by searching: `GET {{baseUrl}}/rest/2.0/assets?name=Customer%20Analytics&nameMatchMode=EXACT`. Save it as a Postman variable `assetId`.

2a. All-time totals

This endpoint ignores the date range and filters; it always answers for the asset's whole history.

```
GET {{usage}}/assets/{{assetId}}/summary
```

```
{
  "totalVisitCount": 25,
  "uniqueVisitorCount": 2,
  "firstVisitDate": "2026-09-29"
}
```

`uniqueVisitorCount` counts distinct people, so it is the better signal of reach: 25 visits from 2 people is a very different story from 25 visits from 20.

2b. Visit trend in your range

Note the lowercase `timeseries` in this path; it differs from the content-wide endpoints.

```
GET {{usage}}/assets/{{assetId}}/visits/timeseries?startDate=
{{startDate}}&endDate={{endDate}}&granularity=Week
```

Each row has the bucket dates and a `count`. Our example asset shows 1, 10 and 5 visits in three consecutive weeks: it was found, used heavily, then settled.

2c. Who visits it

```
GET {{usage}}/assets/{{assetId}}/visitors/top?startDate=
{{startDate}}&endDate={{endDate}}&limit=10
```

```
{
  "results": [
    { "userId": "9693d5ce-9fb4-4e97-b46e-7218526eda14", "visits": 15 },
    { "userId": "01a06913-ca5f-733c-bb50-065eeb3aabc9", "visits": 1 }
  ]
}
```

The response gives user IDs only. To turn them into names, call the public users API for each one:

```
GET {{baseUrl}}/rest/2.0/users/9693d5ce-9fb4-4e97-b46e-7218526eda14
```

and read `firstName`, `lastName` and `userName` from the response.

Try it

1. Pick the top asset from Lesson 1c and run 2a to 2c for it.
2. Find an asset you suspect is unused. A summary with `totalVisitCount` of 0 and no `firstVisitDate` means nobody has opened it since Usage Analytics started collecting.

Lesson 3: Who is using Collibra, and do they come back?

Goal: measure adoption with the users service: how many people are active, who the heaviest users are, how intensely people use Collibra, and whether they return.

3a. Active, inactive and new users

The users summary returns one number per call. Choose the population with `summaryUserType` and make three calls:

```
GET {{users}}/summary?startDate={{startDate}}&endDate=
{{endDate}}&summaryUserType=Active
GET {{users}}/summary?startDate={{startDate}}&endDate=
{{endDate}}&summaryUserType=Inactive
GET {{users}}/summary?startDate={{startDate}}&endDate=
{{endDate}}&summaryUserType=New
```

```
{ "count": 4 }
```

summaryUserType	Counts
`Active`	Distinct people who signed in at least once in the range
`Inactive`	Known users with no sign-in in the range
`New`	People whose first-ever sign-in falls in the range

`Active` here is deduplicated: one person who signs in every day is still 1. Keep that in mind in 3c.

3b. Most active users

```
GET {{users}}/top?startDate={{startDate}}&endDate={{endDate}}&limit=10
```

```
{
  "results": [
    { "id": "9693d5ce-9fb4-4e97-b46e-7218526eda14", "fullName": "Jane
Analyst", "count": 131 }
  ]
}
```

Unlike the asset visitors in Lesson 2c, this list already includes `fullName`. `count` is the person's visits across all content types.

3c. Adoption trends

Three time-series endpoints break active users down by a `category`. They share the same request shape:

```
GET {{users}}/userRetention/timeSeries?startDate={{startDate}}&endDate=
{{endDate}}&granularity=Week
```

Endpoint	`category` values	Extra parameter
`usageRate/timeSeries`	`High`, `Medium`, `Low`	`granularity` must be `Week`, `Month` or `Quarter`
`userRetention/timeSeries`	`Acquired`, `Retained`, `Returning`	None
`licenseTypes/timeSeries`	Your license types, such as `Creator`	`userType` = `Active`, `Inactive` or `New`

Retention categories mean:

- **Acquired:** signed in for the first time ever in this bucket.
- **Retained:** active in this bucket and in the previous one.
- **Returning:** active in this bucket and at some point before, but not in the previous bucket.

A weekly retention response for a new rollout might read 4 Acquired in week 39, then 3 Retained in week 40, then 2 Retained and 1 Returning in week 41. That says all four new users stuck around.

Do not add buckets together to count people. Each bucket counts a person once, so summing four weekly buckets gives *user-weeks*, not users. For distinct people over the whole range, use 3a.

Try it

1. Run 3a for this month and last month. Is `Active` growing?
2. Run `usageRate/timeSeries` with `granularity=Day`. Usage rate has no daily buckets, so expect it to be rejected; switch to `Week`.
3. Run `licenseTypes/timeSeries` with `userType=Active&granularity=Month` to see which license types your active users hold.

Lesson 4: Narrow it down with filters

Goal: run any call from Lessons 1 to 3 for one audience (a team, a role, a license type) or one slice of the catalog (a community, an asset type).

Filters take IDs, not names, so this is always a two-step job: look up the valid values, then pass their IDs.

4a. Look up valid filter values

Each filter has a `/list` endpoint on the content service. The lists are scoped to the date range, so you only see values that had activity.

```
GET {{usage}}/groups/list?startDate={{startDate}}&endDate=
{{endDate}}&limit=100&offset=0
GET {{usage}}/roles/list?startDate={{startDate}}&endDate=
{{endDate}}&limit=100&offset=0
GET {{usage}}/organizations/list?startDate={{startDate}}&endDate=
{{endDate}}&limit=100&offset=0
GET {{usage}}/assetTypes/list?startDate={{startDate}}&endDate=
{{endDate}}&limit=100&offset=0
GET {{usage}}/licenseTypes/list
```


```
{
  "results": [
    { "id": "00000000-0000-0000-0001-000100000001", "label": "Business
Analysts Community", "organizationType": "Community" },
    { "id": "00000000-0000-0000-0001-000100000004", "label": "91. AI
Governance", "organizationType": "Community" }
  ]
}
```


Lists return up to 100 rows per call. If you get exactly 100 back, raise `offset` by 100 and call again until a page comes back short. License types are the exception: they have no ID, so you filter on the `label` itself.

4b. Apply the filter

To filter by…	List endpoint	Parameter to send	Value	Services
User group	`groups/list`	`userGroupIds`	`id`	Content and users
Role	`roles/list`	`userRoleIds`	`id`	Content and users
License type	`licenseTypes/list`	`userLicenseTypes`	`label`	Content and users
Community or domain	`organizations/list`	`organizationIds`	`id`	Content only
Asset type	`assetTypes/list`	`assetTypeIds`	`id`	Content only

For example, the top 10 assets visited by members of a *Data Stewards* group, limited to Business Terms:

```
GET {{usage}}/visits/top?startDate={{startDate}}&endDate=
{{endDate}}&visitType=Asset&limit=10&granularity=Day
    &userGroupIds=019abc49-f08c-7cf8-bb83-3bc3db220c0c
    &assetTypeIds=00000000-0000-0000-0000-000000011001
```

(Shown on several lines for reading; in Postman it is one URL, or one row per parameter on the **Params** tab.)

Rules worth remembering:

- **Different filters combine with AND:** the group *and* the asset type.
- **Repeating the same filter means OR:** two `userGroupIds` match members of either group.
- **Content filters do nothing on the users service.** `organizationIds` and `assetTypeIds` apply only to the content service; leave them off `{{users}}` calls, because a sign-in is not tied to any part of the catalog.

Try it

1. Find your largest community with `organizations/list`, then rerun Lesson 1a for just that community.
2. Rerun Lesson 3a for one user group. What share of the group signed in this month?

Putting it together: recipes

The lessons give you single answers. These recipes combine them into the reports people actually ask for.

Recipe 1: This month vs. last month

The API returns one period per call, so a comparison is two calls with the same parameters and different dates. For example, September vs. August:

```
GET {{usage}}/visits/summary?startDate=2026-09-01&endDate=2026-09-30
GET {{usage}}/visits/summary?startDate=2026-08-01&endDate=2026-08-31
```

Then compute the change yourself:

```
\text{change \%} = \frac{\text{current} - \text{previous}}{\text{previous}}
\times 100
```

If the previous period is 0, the change is undefined; report it as *new* rather than as an infinite percentage. The same recipe works for Active users (Lesson 3a): compare like with like, ideally periods of the same length.

Recipe 2: The complete list, beyond 100 rows

`/top` stops at 100. The download endpoints return every row as CSV:

```
GET {{usage}}/visits/timeSeries/download?startDate={{startDate}}&endDate=
{{endDate}}&visitType=Asset&granularity=Quarter
Accept: text/csv
```

In Postman, use the arrow next to **Send** and choose **Send and Download** to save the file. Use `granularity=Quarter` (or the largest bucket that covers your range) to get roughly one row per item; smaller buckets give one row per item per bucket, and you add up the `Count` column yourself.

Download endpoint	One row per	Useful columns
`{{usage}}/visits/timeSeries/download`	Item per bucket	Item name and Id, `Asset Type`, `Domain`, `Community`, `Community Hierarchy`, `Count`
`{{users}}/licenseTypes/timeSeries/download` (needs `userType`)	User per bucket	`User`, `Required License`, `Roles`, `Number of Sessions`, `Asset Visits`, `Domain Visits`, … `Interval Start Date`
`{{users}}/usageRate/timeSeries/download`	User per bucket	`User`, `Usage Rate`, `Usage Days`, `Interval Start Date`
`{{users}}/userRetention/timeSeries/download`	User per bucket	`User`, `Category`, `Interval Start Date`

The content download is how you build a per-community or per-asset-type breakdown: open it in Excel and pivot on `Community` or `Asset Type`.

Recipe 3: A one-click monthly report in Postman

Let Postman work out "last month" so nobody edits dates by hand. Paste this into the collection's **Scripts > Pre-request** tab:

```
// Sets startDate/endDate to the previous calendar month (UTC)
const now = new Date();
const first = new Date(Date.UTC(now.getUTCFullYear(), now.getUTCMonth() - 1,
1));
const last  = new Date(Date.UTC(now.getUTCFullYear(), now.getUTCMonth(), 0));
const iso = d => d.toISOString().slice(0, 10);
pm.collectionVariables.set("startDate", iso(first));
pm.collectionVariables.set("endDate", iso(last));
```

Collection variables override the environment values from Step 1 while the collection runs; delete them to go back to fixed dates.

Then add a check to the **Scripts > Post-response** tab of the *visits/summary* request, so every run tells you the total and fails loudly if access breaks:

```
pm.test("Status is 200", () => pm.response.to.have.status(200));
const rows = pm.response.json().results || [];
const total = rows.reduce((s, r) => s + Number(r.count || 0), 0);
console.log(`Visits ${pm.collectionVariables.get("startDate")} to
${pm.collectionVariables.get("endDate")}: ${total}`);
pm.collectionVariables.set("totalVisits", total);
```

Put the requests for your report (Lessons 1a, 1c, 3a, 3b) in one folder and run the folder with **Run collection**. Scheduled runs in Postman, or the Newman command-line runner, turn this into an automated monthly job.

Recipe 4: Beyond Postman

Everything above is plain HTTP, so the same calls work from any language. A minimal Python version of Lesson 1a:

```
import os, requests

BASE = "https://your-company.collibra.com/rest/usageAnalyticsUsage/v1"
auth = (os.environ["COLLIBRA_USER"], os.environ["COLLIBRA_PASS"])

r = requests.get(f"{BASE}/visits/summary",
                 params={"startDate": "2026-09-01", "endDate": "2026-09-30",
                         "excludeAdmin": "true"},
                 auth=auth, headers={"Accept": "application/json"},
timeout=60)
r.raise_for_status()
for row in r.json()["results"]:
    print(f'{row["visitType"]:<10} {row["count"]:>6}')
```

`requests` sends a list value as a repeated parameter, so `params={"userGroupIds": [id1, id2]}` produces the OR filter from Lesson 4.

Troubleshooting and FAQ

Symptom	Likely cause	Fix
`401 Unauthorized`	Wrong credentials, or no environment selected in Postman	Check the environment drop-down and the `username` / `password` values
`403 Forbidden`	User lacks the Usage Analytics (Insights) permission	Ask an administrator to grant a global role that includes Insights
`404 Not Found` on every call	Typo in `baseUrl`, a trailing slash, or Usage Analytics not enabled	Compare with the URL in your browser; confirm *Insights > Usage Analytics* exists
`404` on one call only	Path typo or wrong case	Paths are case-sensitive: `timeSeries` for content-wide calls, `timeseries` for per-asset calls
`400 Bad Request`	Bad date format, end before start, or an unsupported value	Dates are `YYYY-MM-DD`; `usageRate` needs `Week`, `Month` or `Quarter`
Today's activity is missing	Data is aggregated periodically	Check `lastRefreshed`; end ranges on yesterday
Fewer buckets than expected	Empty buckets are omitted	Fill missing buckets with zero before charting
Numbers stopped matching after an upgrade	The internal API changed	Compare with the Usage Analytics screens and your browser's developer tools (below)
Results identical with and without a filter	Filter sent to a service that does not use it, or misspelled parameter	Content filters only work on `{{usage}}`; check the parameter name

Why don't my numbers match the Usage Analytics screen? Check three things: the same date range (both ends inclusive), the same *exclude admins* / *exclude disabled users* settings, and the same filters. The screen and the API read the same data, so with identical inputs they agree.

Why is "Active users" lower than the sum of my weekly chart? Because the summary counts each person once, while each weekly bucket counts them again. See Lesson 3c.

How do I see exactly which call the Collibra UI makes? Open *Insights > Usage Analytics* in Chrome or Edge, press **F12**, open the **Network** tab and filter on `usageAnalytics`. Every chart's request appears there with its full URL and parameters. Right-click a request and choose **Copy > Copy as cURL**, then in Postman use **Import > Raw text** to turn it into a request. This is also the quickest way to see what changed after a Collibra upgrade.

Can I use these endpoints to change data? No. Everything in this guide is read-only.

Is there a rate limit? None is documented. Be considerate: run large downloads off-hours, and keep scripts to a handful of requests at a time.

Appendix: endpoint quick reference

All endpoints are `GET`. *Range* means `startDate` and `endDate` are required; *Filters* means the audience filters (`userGroupIds`, `userRoleIds`, `userLicenseTypes`, `excludeAdmin`, `excludeDisabledUser`) apply, plus `organizationIds` and `assetTypeIds` on the content service.

Content service: `/rest/usageAnalyticsUsage/v1`

Path	Extra parameters	Returns	Lesson
`/lastRefreshed`	None	`lastRefreshDateTime`	Step 1
`/visits/summary`	Range, filters	`results[]`: `visitType`, `count`	1a
`/visits/timeSeries`	Range, filters, `granularity`	`results[]`: bucket dates, `visitType`, `count`	1b
`/visits/top`	Range, filters, `visitType`, `limit`, `granularity`	`results[]`: `id`, `label`, `count`	1c
`/visits/timeSeries/download`	Range, filters, `visitType`, `granularity`	CSV, one row per item per bucket	Recipe 2

Path	Extra parameters	Returns	Lesson
`/assets/{id}/summary`	None	`totalVisitCount`, `uniqueVisitorCount`, `firstVisitDate`	2a
`/assets/{id}/visits/timeseries`	Range, filters, `granularity`	`results[]`: bucket dates, `count`	2b
`/assets/{id}/visitors/top`	Range, filters, `limit`	`results[]`: `userId`, `visits`	2c
`/groups/list`, `/roles/list`, `/organizations/list`, `/assetTypes/list`	Range, `limit`, `offset`	`results[]`: `id`, `label` (+ `organizationType`)	4a
`/licenseTypes/list`	None	`results[]`: `label`	4a

Users service: `/rest/usageAnalyticsUsers/v1`

Path	Extra parameters	Returns	Lesson
`/summary`	Range, filters, `summaryUserType`	`count`	3a
`/top`	Range, filters, `limit`	`results[]`: `id`, `fullName`, `count`	3b
`/usageRate/timeSeries`	Range, filters, `granularity` (not `Day`)	`results[]`: bucket dates, `category`, `count`	3c
`/userRetention/timeSeries`	Range, filters, `granularity`	`results[]`: bucket dates, `category`, `count`	3c
`/licenseTypes/timeSeries`	Range, filters, `granularity`, `userType`	`results[]`: bucket dates, `category`, `count`	3c
`.../timeSeries/download` (each of the three above)	Same as its JSON version	CSV, one row per user per bucket	Recipe 2

Public REST endpoints used alongside: `GET /rest/2.0/assets/{id}` and `GET /rest/2.0/assets?name=` for asset details and lookup, `GET /rest/2.0/users/{id}` for user names. These are documented in Collibra's REST Core API reference.

Further reading

- Experimental Collibra Usage Analytics dashboard: a complete, open-source dashboard built on every endpoint in this guide. Its `usage-dashboard.js` is a working reference for each call, and every chart's ⓘ button names the endpoint behind it.
- Postman: Scripting and Newman for automating Recipe 3.